

TimePix Centroiding

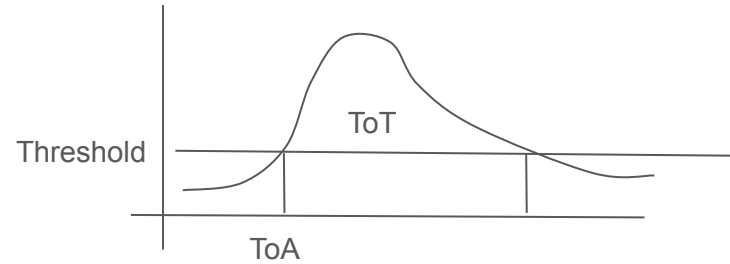
With GPU Speedup

Varian 247 Lab, Stanford University

How we get the data (serval)

- We use the Python “requests” library to send and receive commands from the Timepix via Serval
- For pump-probe scans, we use the computer that controls the stage to stop Serval while the stage is moving and start it collecting again once we have arrived at the next delay

TimePix File Format



- Each pixel reads out independently, giving four values each time:
- (ToA, ToT, Y, X), where ToA is time of arrival, ToT is time over threshold, and Y and X are the spatial coordinates of the pixel
- Pixels generally report once they go back below threshold
- Trigger lines are filled with -1 for ToA, Y, X

```
0.0014267890625 50 396 112
0.00142680625 50 400 112
0.0014267375 175 399 115
0.00142676875 75 396 113
0.0014267625 150 400 113
0.0014267375 200 398 112
0.00142675 175 397 112
0.0014267625 125 400 114
0.0014267078125 675 398 113
0.001426725 225 397 113
0.001426775 75 400 115
0.0014267078125 625 398 114
0.0014267234375 375 397 114
0.0014267375 225 398 115
0.0014267609375 125 397 115
0.00142675 175 399 112
0.0014267109375 425 399 113
0.0014267125 400 399 114
0.00169150676879883 -1 -1 -1
0.0016950921875 50 409 45
0.0016950296875 450 410 45
0.0016950796875 150 413 44
0.0016950875 150 409 43
```

Conversion from .tpx3 to .txt files in C++

Loop through packets

```
for (unsigned long long i = 0; i < NumofPacket; i++) {  
    ...  
  
}
```

Time looping fix

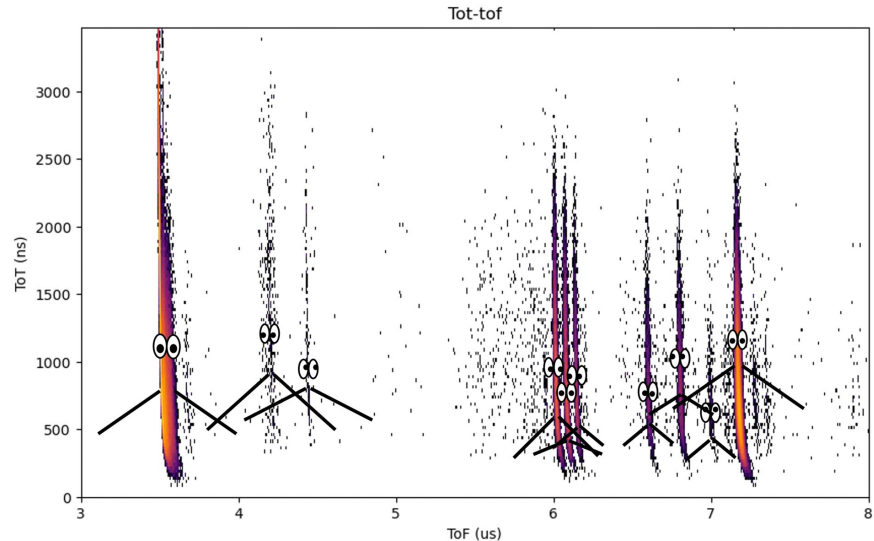
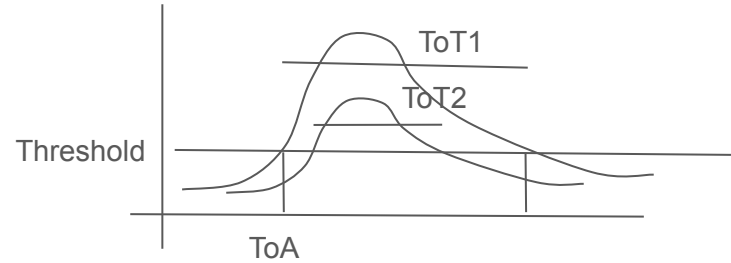
```
int rollover_counter = 0;  
if (TDC_timestamp < prev_TDC_timestamp) { rollover_counter++; // Increment the  
    rollover counter }
```

Removing dark counts

```
if (((global_timestamp / 1E9) - (prev_TDC_timestamp + rollover_counter * time_unit *  
(1ULL<<44) * 1E-9)) < 0.0001) {  
    xy_file << setprecision(15) << global_timestamp / 1E9 << " " << TOTns << " " <<  
    x << " " << y << endl; //toa, tot, x and y data can be saved into txt data  
}
```

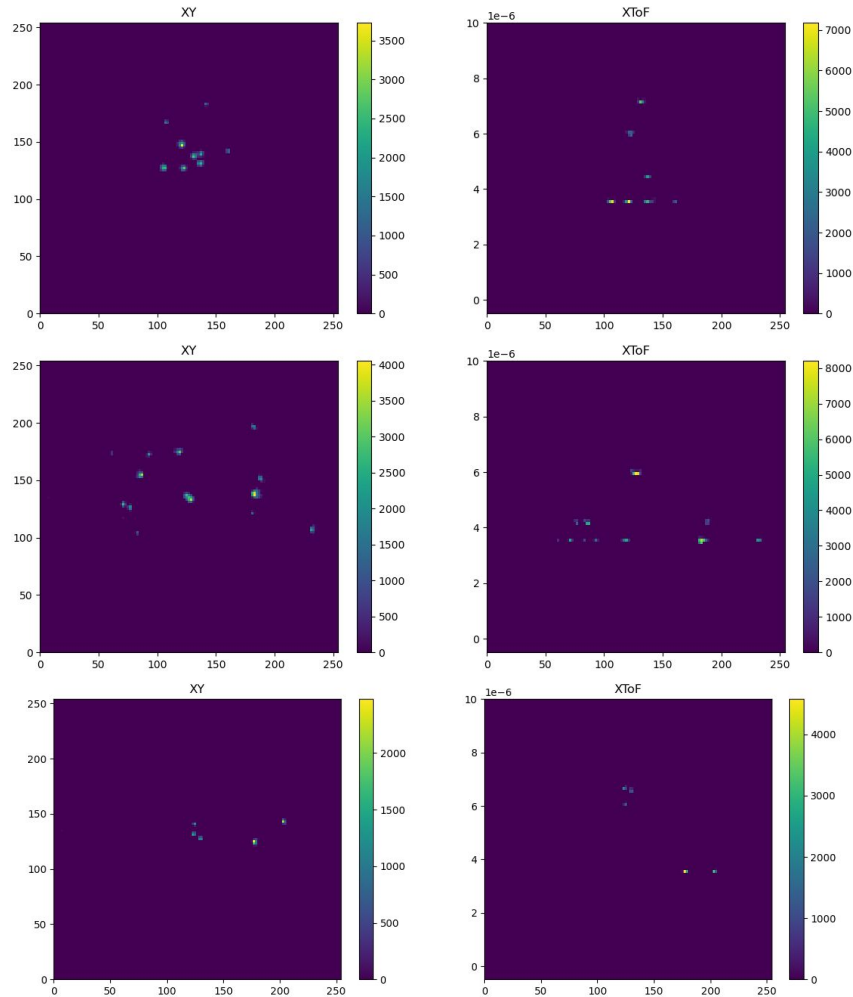
Time over threshold / Time of flight (ToT/ToF) correction

- Pixels that are not as bright will also take longer to go over threshold
- This produces an apparent correlation between ToF and ToA that must be corrected
- Doing this pre-centroiding increases the temporal precision of the centroids



TimePix Frames

- Individual frames are all pixels that go over threshold in between triggers
- Frames can be built by histogramming pixel X, Y, and ToF and weighting by ToT for brightness
- Here we show frames 0, 10, and 100, an arbitrary choice of frames that we will use for subsequent illustration



Which pixels are next to each other? Finding neighborhoods

Consider the array of x-values:

[20, 21, 90, 92, 18]

There are two obvious neighborhoods:

[20, 21, 90, 92, 18]

Build the repeated matrix and subtract its transpose:

[20, 21, 90, 92, 18]	[20, 20, 20, 20, 20]	[0, 1, 70, 72, -2]
[20, 21, 90, 92, 18]	[21, 21, 21, 21, 21]	[-1, 0, 69, 71, -3]
[20, 21, 90, 92, 18]	- [90, 90, 90, 90, 90]	= [-70, -69, 0, 2, -72]
[20, 21, 90, 92, 18]	[92, 92, 92, 92, 92]	[-72, -71, -2, 0, -74]
[20, 21, 90, 92, 18]	[18, 18, 18, 18, 18]	[2, 3, 72, 74, 0]

And condition on the difference being within a certain range, say +/- 5

[0, 1, 70, 72, -2]	[1, 1, 0, 0, 1]
[-1, 0, 69, 71, -3]	[1, 1, 0, 0, 1]
[-70, -69, 0, 2, -72]	→ [0, 0, 1, 1, 0]
[-72, -71, -2, 0, -74]	[0, 0, 1, 1, 0]
[2, 3, 72, 74, 0]	[1, 1, 0, 0, 1]

Do this in three dimensions, X, Y, and ToF to find adjacent pixels for further analysis

Which clusters of pixels should we centroid? Finding local maxima

Consider the ToT value array:

[50, 200, 300, 25, 100, 75]

And the neighborhoods:

[1, 1, 1, 0, 0, 0] and [0, 0, 0, 1, 1, 1]

[1, 1, 1, 0, 0, 0] [50, 200, 300, 25, 100, 75]

[1, 1, 1, 0, 0, 0] [50, 200, 300, 25, 100, 75]

[1, 1, 1, 0, 0, 0] [50, 200, 300, 25, 100, 75]

[0, 0, 0, 1, 1, 1] [50, 200, 300, 25, 100, 75]

[0, 0, 0, 1, 1, 1] [50, 200, 300, 25, 100, 75]

[0, 0, 0, 1, 1, 1] [50, 200, 300, 25, 100, 75]

Multiply the repeated array by the neighborhood and subtract its transpose:

[50, 200, 300, 0, 0, 0] [50, 50, 50, 0, 0, 0] [0, 150, 250, 0, 0, 0]

[50, 200, 300, 0, 0, 0] [200, 200, 200, 0, 0, 0] [-150, 0, 100, 0, 0, 0]

[50, 200, 300, 0, 0, 0] - [300, 300, 300, 0, 0, 0] = **[-250, -100, 0, 0, 0, 0]**

[0, 0, 0, 25, 100, 75] [0, 0, 0, 25, 25, 25] [0, 0, 0, 0, 75, 50]

[0, 0, 0, 25, 100, 75] [0, 0, 0, 100, 100, 100] **[0, 0, 0, -75, 0, -25]**

[0, 0, 0, 25, 100, 75] [0, 0, 0, 75, 75, 75] [0, 0, 0, -50, 25, 0]

Local maxima are rows where the maximum value is zero

(Credit to Chuan for suggesting this)

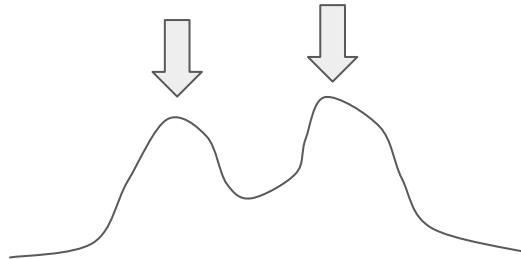
Centroiding in the neighborhood of local maxima

```
centroids_x = (sum(xs*neighbors*tots)/sum(tots*neighbors))[local_maxima_filter]
```

```
centroids_y = (sum(ys*neighbors*tots)/sum(tots*neighbors))[local_maxima_filter]
```

```
centroids_t = tofs[local_maxima_filter]
```

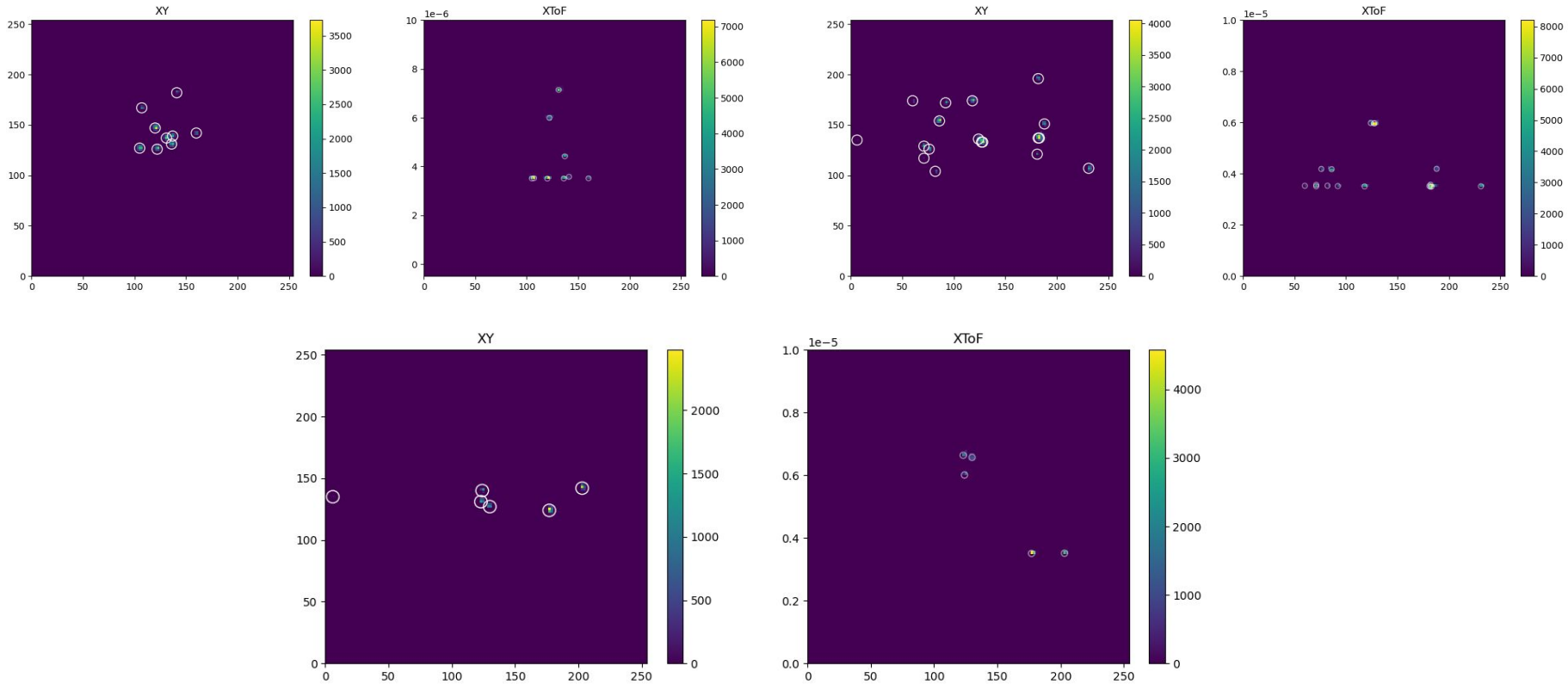
This performs the centroid in the neighborhood of each pixel, but then only selects those centroids that were taken around the local maximum. This selects the correct centroids for the scenario:



Centroiding summary: three key steps

1. Find the neighbors of each pixel in the trigger by building the adjacency matrix
2. Find the local maxima in each neighborhood using the adjacency matrix and some math tricks
3. Centroid around the local maxima to output a list of (xcent, ycent, tofcent) values

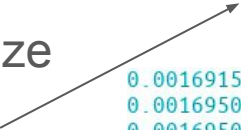
Centroiding results: three representative frames



Batch processing: using GPU parallelization

- Each step in the process is composed of simple array operations that can be moved to the GPU and parallelized
- Approach: process multiple triggers simultaneously by adding a trigger axis to the arrays and letting PyTorch parallelize across this axis

Parallelize



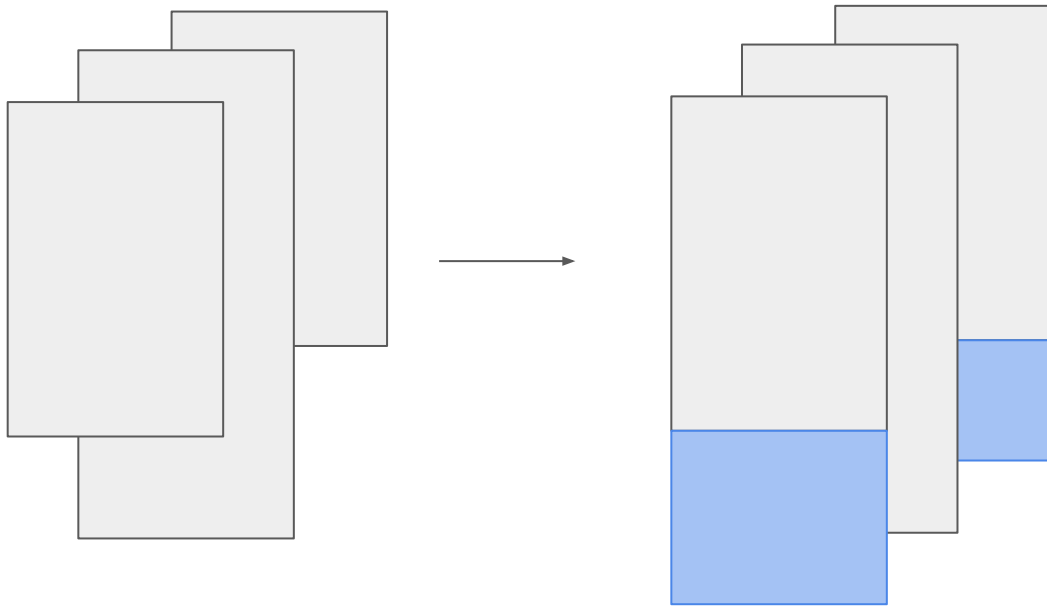
```
0.00269141588134766 -1 -1 -1
0.002694965625 125 381 180
0.002694978125 75 377 179
0.00169150676879883 -1 -1 -1
0.0016950921875 50 409 45
0.0016950296875 450 410 45
0.0014267890625 50 396 112
0.00142680625 50 400 112
0.0014267375 175 399 115
0.00142676875 75 396 113
0.0014267625 150 400 113
0.0014267375 200 398 112
0.00142675 175 397 112
0.0014267625 125 400 114
0.0014267078125 675 398 113
0.001426725 225 397 113
0.001426775 75 400 115
0.0014267078125 625 398 114
0.0014267234375 375 397 114
0.0014267375 225 398 115
0.0014267609375 125 397 115
```

0.00269141588134766 -1 -1 -1
0.002694965625 125 381 180
0.002694978125 75 377 179
0.00169150676879883 -1 -1 -1
0.0016950921875 50 409 45
0.0016950296875 450 410 45
0.0014267890625 50 396 112
0.00142680625 50 400 112
0.0014267375 175 399 115
0.00142676875 75 396 113
0.0014267625 150 400 113
0.0014267375 200 398 112
0.00142675 175 397 112
0.0014267625 125 400 114
0.0014267078125 675 398 113
0.001426725 225 397 113
0.001426775 75 400 115
0.0014267078125 625 398 114
0.0014267234375 375 397 114
0.0014267375 225 398 115
0.0014267609375 125 397 115

Allocating memory: finding the largest trigger

Triggers have different numbers of pixels, but the parallelization on the GPU requires rectangular arrays

Solution: find the largest trigger, and zero-pad the other triggers (this does nothing to the centroids)



Functions for batched neighbors and local maxima

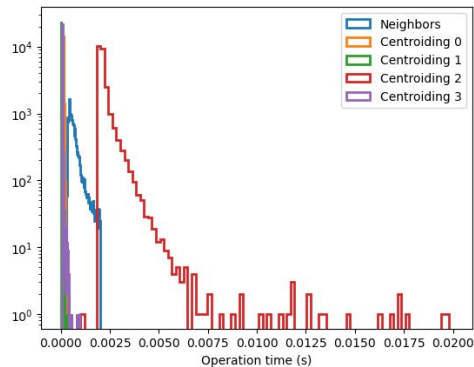
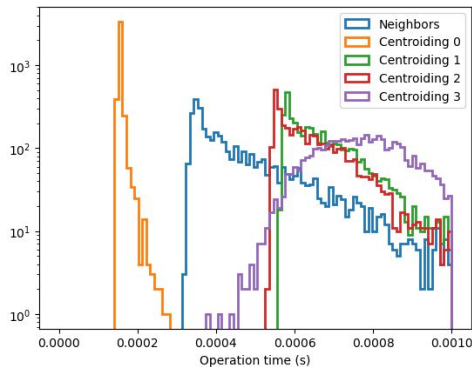
```
def get_neighbors(block,size=3,tsep=0.5e-6):
    block_rep = block.expand(block.shape[0],[ -1 for _ in range(len(block.shape))])
    block_sub = block_rep - block_rep.transpose(1,0)
    return
    (torch.abs(block_sub[:, :, 3])<=size)&(torch.abs(block_sub[:, :, 2])<=size)&(torch.abs(block_sub[:, :, 0])<
    =tsep)

def get_local_maxima_batch(block,neighbors,min_size=3,**kwargs):
    try:
        block_rep = block.expand(block.shape[0],[ -1 for _ in range(len(block.shape))])
        tot_neighbors = block_rep[:, :, 1]*neighbors
        tot_neighbors_sub = tot_neighbors.transpose(1,0) - torch.diagonal(tot_neighbors).T
        local_maxima =
    (torch.amax(tot_neighbors_sub,dim=0)==0)&(torch.amin(tot_neighbors_sub,dim=0)<0)&(torch.sum(neighbors
    ,dim=0)>=min_size)
        return local_maxima
    except Exception as e:
        return torch.tensor([[]],dtype=int).to(block.device)
```

Picking local maxima efficiently: boolean indexing vs. array indexing

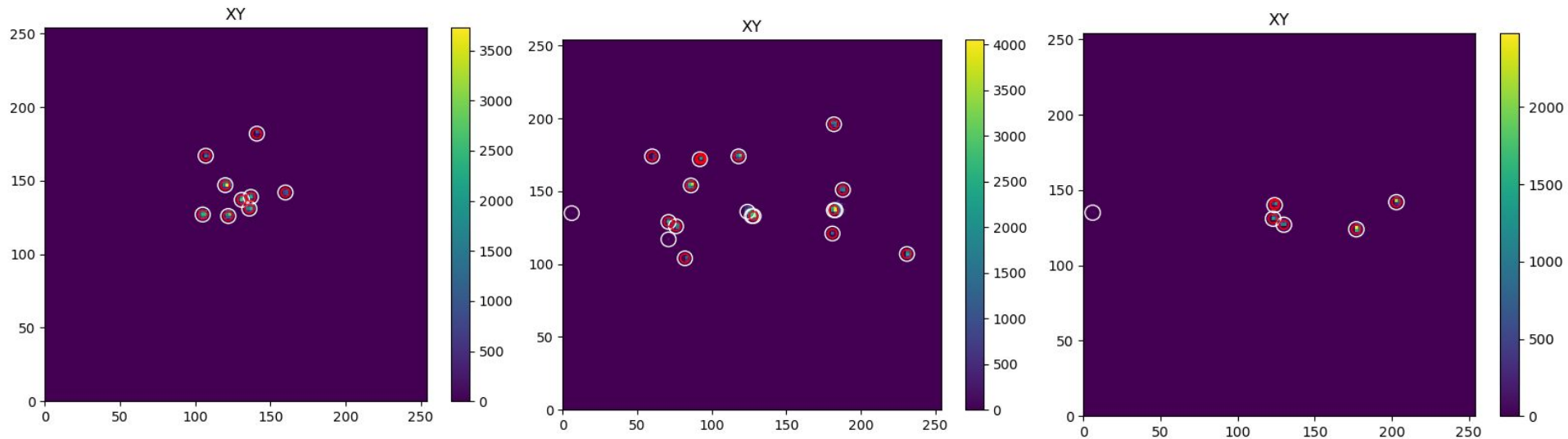
- Local maxima function returns a boolean array indicating locations of local maxima - but this is slow to evaluate once you have it
- Instead: use the boolean array once to find the array indices, and then use array indexing to pick out the local maxima

Speed comparisons for two methodologies - look at red histograms



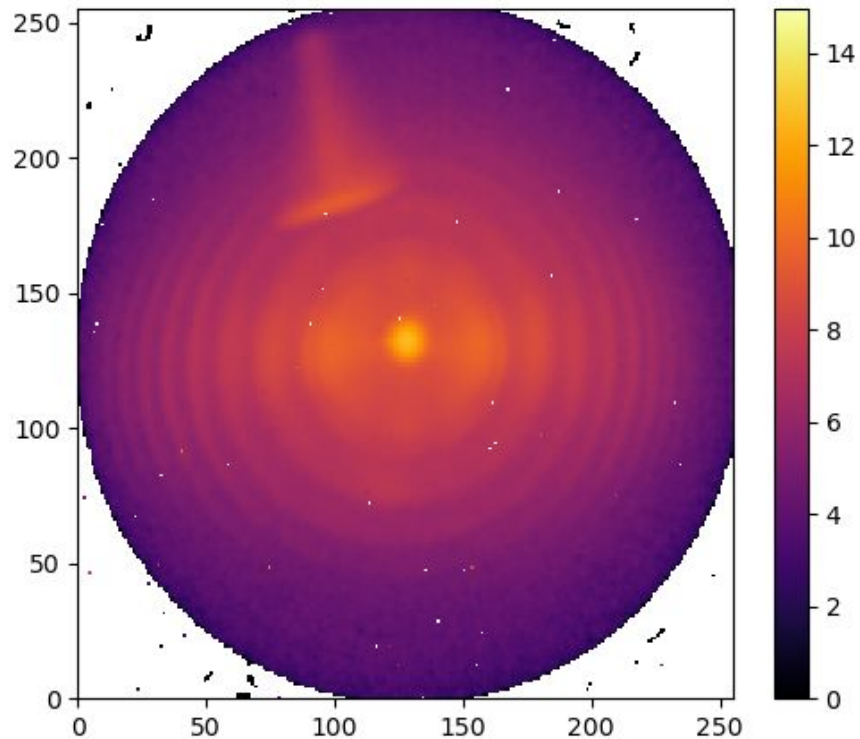
Comparing batched and unbatched centroids performance

Unbatched are shown in white, batched in red



Comparing centroids to raw pixel data

Raw Data



Centroids

